

Data Analysis with Busiest Airports and Monte Carlo Integration

oyindasanni

2026-04-01

Busiest Airports Analysis

The table and line plot tell a story about how global air travel collapsed and then had a return in the wake of the COVID pandemic. From the table, every airport saw a drop in passengers in 2020; ATL fell from the pre-pandemic levels to 42.9 million, while more international airports like LHR and DXB were hit harder and dropped. This makes sense since international travel restrictions were worse than the domestic ones; this is why ATL was able to recover faster than others. By 2024- 2025, most airports had recovered and passed the pre Covid levels, with DXB reaching 95.2 million passengers in 2025. PKX dropped while others were climbing in 2022, which shows their zero COVID policy, before they eventually surged to 53.6 by 2025. Together, both visualizations show that geography and dependence on international versus domestic travel show not only how the airports were hit but also how quickly they were able to bounce back.

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

`filter`, `lag`

The following objects are masked from 'package:base':

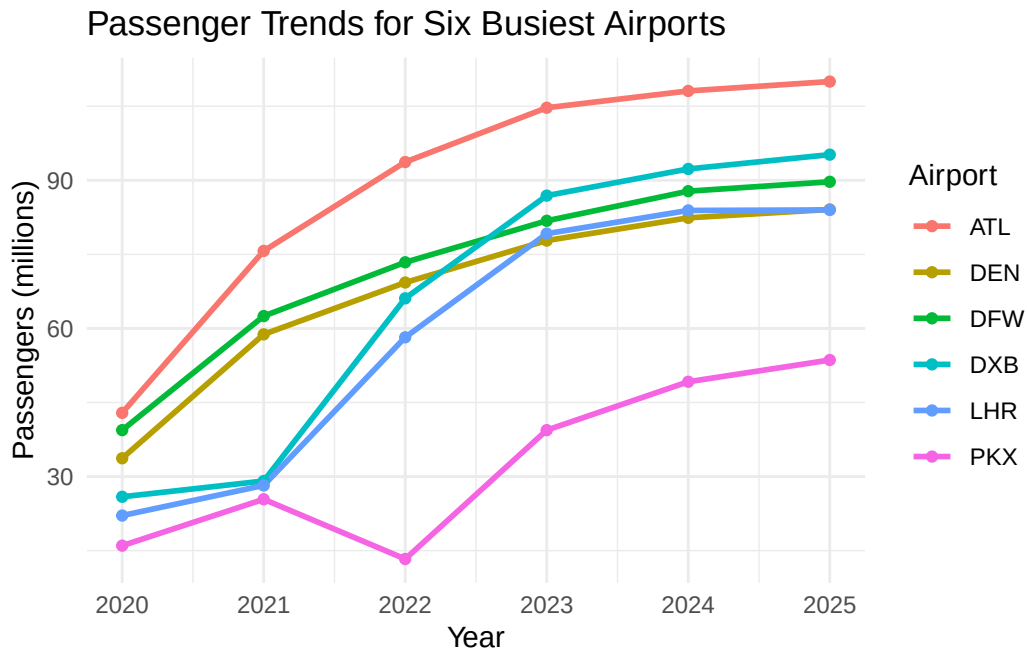
`intersect`, `setdiff`, `setequal`, `union`

Airport Summary Table

Table 1: Total Passengers (in millions) for Six Busiest Airports (2020–2025)

Airport	2020	2021	2022	2023	2024	2025
ATL	42.9	75.7	93.7	104.7	108.1	110.0
DFW	39.4	62.5	73.4	81.8	87.8	89.7
DEN	33.7	58.8	69.3	77.8	82.4	84.1
DXB	25.9	29.1	66.1	86.9	92.3	95.2
LHR	22.1	28.2	58.2	79.2	83.9	84.0
PKX	16.0	25.4	13.3	39.4	49.2	53.6

Airport Visualization



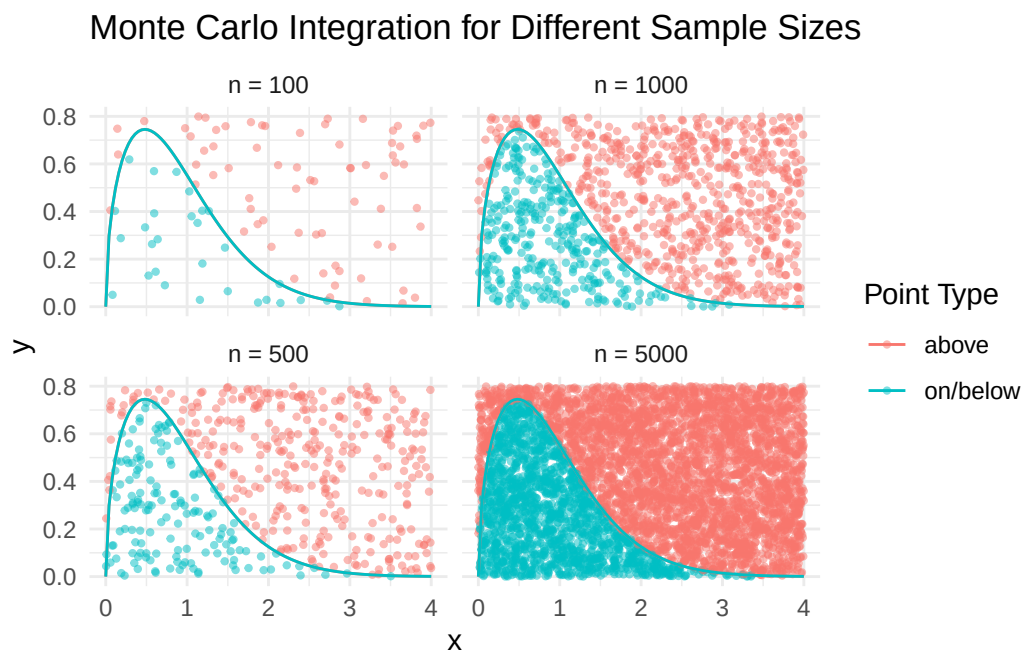
Monte Carlo Numerical Integration

The small plot shows how the Monte Carlo simulation gets more accurate as the sample points increase. At $n=10$, the estimates have high variability, so only a few points are used to approximate the area. At $n=10,000$, the points fill up more, and the estimate stabilizes, so we have a more reliable result.

The exact value of the integral of the Weibull probability density function with shape = 1.5 and scale = 1 from 0 to 4 is very close to 1, since a probability density function always integrates to 1 over its full range, and 0 to 4 captures nearly all of the distribution. The small multiple supports this, as n increases, the estimated integral approaches approximately 1, with the $n = 10000$ panel giving the closest estimate. This shows that while Monte Carlo integration is random and imprecise at low resolutions, it converges toward the true answer as resolution increases.

Small Multiple Visualization

```
Warning: Multiple drawing groups in `geom_function()`  
i Did you use the correct group, colour, or fill aesthetics?  
Multiple drawing groups in `geom_function()`  
i Did you use the correct group, colour, or fill aesthetics?  
Multiple drawing groups in `geom_function()`  
i Did you use the correct group, colour, or fill aesthetics?  
Multiple drawing groups in `geom_function()`  
i Did you use the correct group, colour, or fill aesthetics?
```



Planning and Prompting GenAI Tools

My Plan

Goal: The goal is to tidy the calcium.csv dataset, wrangle it to a form for visualization and then create a plot comparing ulnar calcium levels overtime between women in the null treatment group and those in the diet or exercise group.

Needs: Noun: calcium.csv, readr, dplyr, tidyr and ggplot2 Verb: Select, Read, Rename, Add, Pivot, Bind, plot

Steps: 1. Read the data: load the calcium.csv using read.csv and inspect to confirm the structure matches expectations. 2. Split by treatment group: separate the data into two data frames. Column 1-4 for null group and then Column 5-8 for the diet / exercise group. Rename the columns in both to :Initial, Year1 , Year2, Year3 3. Add identifying variables: Add a subject_id to each data frame and a treatment column 4. Pivot into a long format: Use pivot_longer() on each data frame to convert columns in two columns:year and calcium 5. Bind the two groups together: use bind_rows() to combine the long data frames into a single tidy data frame. 6. Convert the year column to be ordered so the x axis plot in the correct order 7. Create the visualization.Use the ggplot2 to create a line plot with year on the x axis and calcium on the y axis. 8. Add a descriptive title, axis labels and a clean theme.

Plan-Based Prompt Results

New names:

Rows: 16 Columns: 8

-- Column specification

----- Delimiter: "," dbl

(8): y0...1, y1...2, y2...3, y3...4, y0...5, y1...6, y2...7, y3...8

i Use `spec()` to retrieve the full column specification for this data. i

Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

* `y0` -> `y0...1`

* `y1` -> `y1...2`

* `y2` -> `y2...3`

* `y3` -> `y3...4`

* `y0` -> `y0...5`

* `y1` -> `y1...6`

* `y2` -> `y2...7`

* `y3` -> `y3...8`

A tibble: 6 x 8

y0...1 y1...2 y2...3 y3...4 y0...5 y1...6 y2...7 y3...8

	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	87.3	86.9	86.7	75.5	83.8	85.5	86.2	81.2
2	59	60.2	60	53.6	65.3	66.9	67	60.6
3	76.7	76.5	75.7	69.5	81.2	79.5	84.5	75.2
4	70.6	76.1	72.1	65.3	75.4	76.7	74.3	66.7
5	54.9	55.1	57.2	49	55.3	58.3	59.1	54.2
6	78.2	75.3	69.1	67.6	70.3	72.3	70.6	68.6

Rows: 16

Columns: 8

```
$ y0...1 <dbl> 87.3, 59.0, 76.7, 70.6, 54.9, 78.2, 73.7, 61.8, 85.3, 82.3, 68.~
$ y1...2 <dbl> 86.9, 60.2, 76.5, 76.1, 55.1, 75.3, 70.8, 68.7, 84.4, 86.9, 65.~
$ y2...3 <dbl> 86.7, 60.0, 75.7, 72.1, 57.2, 69.1, 71.8, 68.2, 79.2, 79.4, 72.~
$ y3...4 <dbl> 75.5, 53.6, 69.5, 65.3, 49.0, 67.6, 74.6, 57.4, 67.0, 77.4, 60.~
$ y0...5 <dbl> 83.8, 65.3, 81.2, 75.4, 55.3, 70.3, 76.5, 66.0, 76.7, 77.2, 67.~
$ y1...6 <dbl> 85.5, 66.9, 79.5, 76.7, 58.3, 72.3, 79.9, 70.9, 79.0, 74.0, 70.~
$ y2...7 <dbl> 86.2, 67.0, 84.5, 74.3, 59.1, 70.6, 80.4, 70.3, 76.9, 77.8, 68.~
$ y3...8 <dbl> 81.2, 60.6, 75.2, 66.7, 54.2, 68.6, 71.6, 64.1, 70.3, 67.9, 65.~
```

[1] 16

[1] 64

[1] 16

[1] 64

[1] 124

A tibble: 2 x 2

	treatment	n
	<chr>	<int>
1	Diet/Exercise	64
2	Null	60

A tibble: 6 x 4

	subject_id	treatment	year	calcium
	<int>	<chr>	<ord>	<dbl>
1	1	Null	Initial	87.3
2	1	Null	Year1	86.9

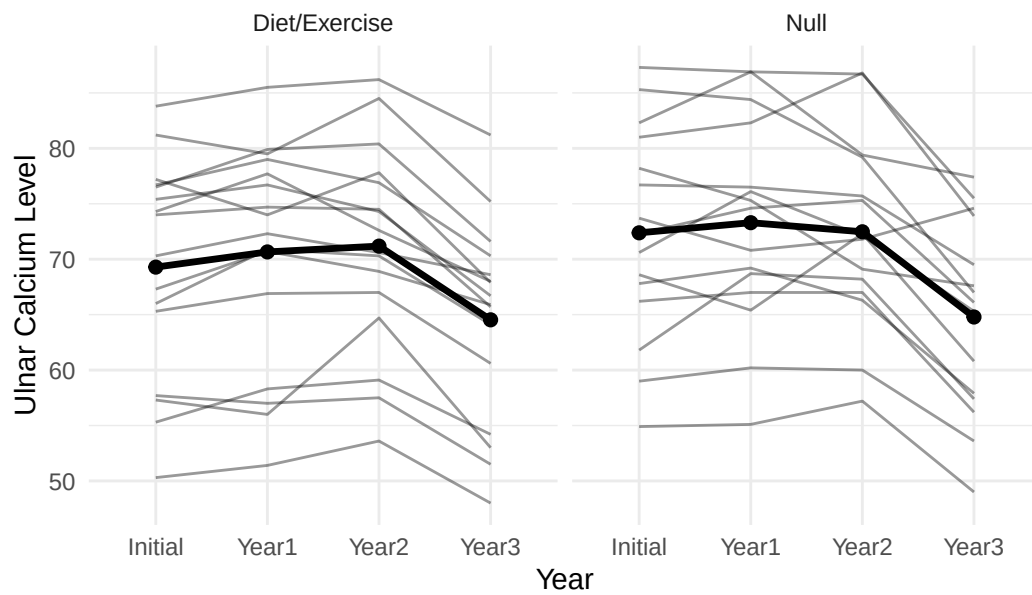
3	1 Null	Year2	86.7
4	1 Null	Year3	75.5
5	2 Null	Initial	59
6	2 Null	Year1	60.2

Rows: 124

Columns: 4

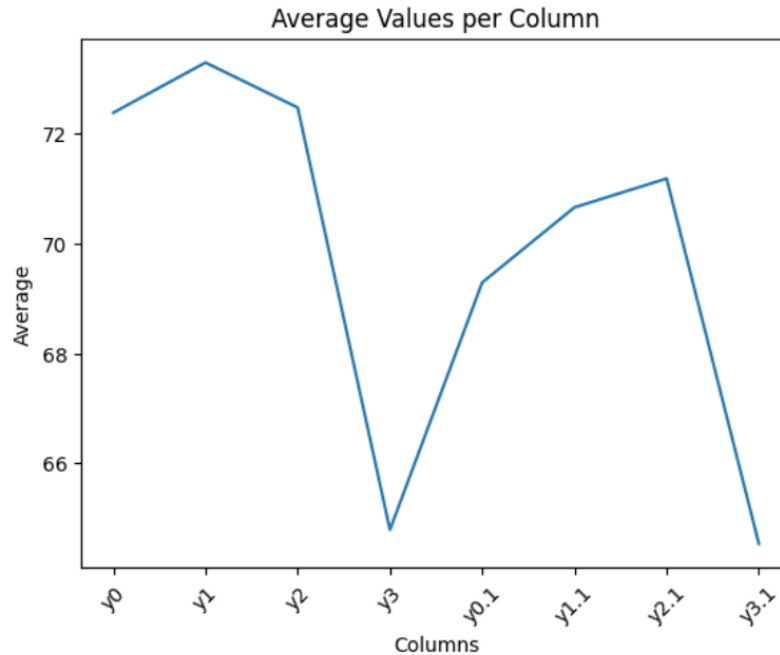
```
$ subject_id <int> 1, 1, 1, 1, 2, 2, 2, 2, 3, 3, 3, 3, 4, 4, 4, 4, 5, 5, 5, 5,~
$ treatment  <chr> "Null", "Null", "Null", "Null", "Null", "Null", "Null", "Nu~
$ year       <ord> Initial, Year1, Year2, Year3, Initial, Year1, Year2, Year3,~
$ calcium    <dbl> 87.3, 86.9, 86.7, 75.5, 59.0, 60.2, 60.0, 53.6, 76.7, 76.5,~
```

Ulnar Calcium Levels Over Time by Treatment Group



Generic Prompt Results

Prompt: Create a data visualization for these data. Calcium.csv



ChatGPT responded with the following: “I created a visualization showing the average value of each column in your dataset. Each point represents the mean (average) of one column (y0, y1, y2, y3, etc.). You can quickly compare which variables tend to have higher or lower values overall. For example, some columns cluster around ~70–73, while others drop closer to ~65. If you want something more insightful, I can also create boxplots, time series plots, correlation heatmaps, or side-by-side comparisons.”

Comparison Narrative

When I first looked at the calcium dataset, I could immediately tell the data were not tidy. The eight columns mixed two different treatment groups together, and the time points were spread across columns instead of being stacked into rows. My plan attended to this directly by splitting the data into two separate groups, adding subject IDs and treatment labels, pivoting to long format, a line plot that showed both individual women’s trajectories and the group mean over time, since that would make it easy to see whether the diet and exercise intervention actually made a difference compared to the null group. The plan-informed ChatGPT response followed my steps closely and produced R code that I felt confident in. It correctly handled the wide-to-long pivot, recognized that the null group had an extra blank row and used `drop_na()` to handle it, and created a faceted `ggplot2` visualization with individual lines and overlaid group means. The code was clean, well-commented, and matched what I had envisioned when writing my plan. The generic prompt response was a completely different story. I simply gave ChatGPT the file with no context, and it had no idea what it was looking at. Rather than

recognizing the longitudinal structure or the two treatment groups, it treated all eight columns as unrelated variables and just plotted the average of each one on a single line. The x-axis labels came out as y0, y1, y2, y3, y0.1, and so on, which are meaningless. The tool even admitted the result might not be very insightful and offered to try something else if I gave it more direction. This activity taught me something important about working with generative AI tools: the quality of the output is almost entirely determined by the quality of the input. When I gave ChatGPT a vague prompt, it gave me a vague and useless result. When I gave it a detailed plan with a clearly stated goal, specific needs, and step-by-step instructions, it produced something genuinely useful. Knowing your data and having a plan before you prompt is not just helpful, it is necessary if you want the tool to actually assist you rather than just generate something that looks like an answer

Reflection

Coming into this course, I had very little experience with R and had never done any kind of formal data analysis. I had heard of things like data wrangling and data visualization before, but I did not really understand what they meant in practice or how to actually do them. Looking back at how far I have come, I can point to several specific things I have genuinely learned. The biggest shift for me has been learning how to think about data structure. Before this course, I would not have noticed that a dataset like `calcium.csv` was not tidy, let alone known what to do about it. Now I can look at a dataset and recognize when columns are doing the job that rows should be doing, and I know how to use `pivot_longer()` to fix it. Being able to separate the calcium data into two treatment groups, add subject IDs, pivot to long format, and bind everything back together felt like a real accomplishment because a few weeks ago I would not have known where to start. I have also grown a lot with `ggplot2`. Early on, making even a basic plot felt confusing because of how the layering system works. Now I can build a plot from scratch, add `facet_wrap()` to split by a grouping variable, overlay summary statistics with `stat_summary()`, and customize labels and themes. The busiest airports analysis was a good example of this — creating a clear, well-labeled bar chart from raw flight data required me to wrangle the data first and then think carefully about how to present it visually. Finally, this assignment specifically taught me a lot about reproducibility and working with Quarto. I had never thought about the idea that an analysis should be something someone else can run and get the same result. Learning to write a document where the narrative, the code, and the output all live together — and where the code is hidden from the main document but available in an appendix — changed how I think about sharing analytical work. It is not just about getting the right answer; it is about communicating the process clearly and making it possible for others to verify and build on what you did

GenAI Usage Appendix

Planning and Prompting Section

- **Tool used:** OpenAI ChatGPT (GPT-5.3)
- **Date:** Tuesday, March 31, 2026

Prompt 1 (Plan-Informed Prompt)

I have a dataset called calcium.csv from a longitudinal study following 31 women over four years, measuring ulnar calcium levels. The first four columns belong to the null treatment group (15 women) and the last four columns belong to the diet/exercise treatment group (16 women), with measurements at Initial, Year 1, Year 2, and Year 3. Please write R code that follows this plan:

Goal: Tidy the calcium.csv dataset, wrangle it into a form suitable for visualization, and create a plot comparing ulnar calcium levels over time between the null and diet/exercise treatment groups.

Needs: We will need calcium.csv, readr, dplyr, tidyr, and ggplot2. We will need to read, select, rename, add, pivot, bind, and plot.

Steps: (1) Read in calcium.csv using read_csv() and inspect with head() and glimpse(). (2) Separate columns 1–4 as the null group and columns 5–8 as the diet/exercise group, renaming both to Initial, Year1, Year2, Year3. (3) Add a subject_id and treatment column to each. (4) Use pivot_longer() to create year and calcium columns — check that rows quadruple. (5) Use bind_rows() to combine both groups into one 124-row data frame — check with count(treatment). (6) Convert year to an ordered factor. (7) Create a ggplot2 line plot with individual trajectories and group mean lines, faceted by treatment. (8) Add a title, axis labels, and a clean theme.

Response: # Goal: # Tidy the calcium.csv dataset, wrangle it into a form suitable for visualization, # and create a plot comparing ulnar calcium levels over time between the # null and diet/exercise treatment groups.

Needs:

calcium.csv, readr, dplyr, tidyr, ggplot2

```
library(readr) library(dplyr) library(tidyr) library(ggplot2)
```

Step 1: Read in the data and inspect it

```
calcium_raw <- read_csv("calcium.csv")  
head(calcium_raw) glimpse(calcium_raw)
```

Step 2: Separate the two treatment groups and rename columns

```
null_group <- calcium_raw |> select(1:4) |> rename( Initial = 1, Year1 = 2, Year2 = 3, Year3  
= 4 )  
diet_exercise_group <- calcium_raw |> select(5:8) |> rename( Initial = 1, Year1 = 2, Year2  
= 3, Year3 = 4 )
```

Step 3: Add subject_id and treatment column to each

```
null_group <- null_group |> mutate( subject_id = 1:n(), treatment = "Null" )  
diet_exercise_group <- diet_exercise_group |> mutate( subject_id = 1:n(), treatment =  
"Diet/Exercise" )
```

Step 4: Pivot each group to long format

```
null_long <- null_group |> pivot_longer( cols = Initial:Year3, names_to = "year", values_to  
= "calcium" )  
diet_exercise_long <- diet_exercise_group |> pivot_longer( cols = Initial:Year3, names_to  
= "year", values_to = "calcium" )
```

Check that rows quadrupled before removing missing values

```
nrow(null_group) nrow(null_long)  
nrow(diet_exercise_group) nrow(diet_exercise_long)
```

Remove missing values from the null group

(there is one extra blank row in the null treatment columns)

```
null_long <- null_long |> drop_na(calcium)
```

Step 5: Bind the two groups together

```
calcium_tidy <- bind_rows(null_long, diet_exercise_long)
```

Check total number of rows and counts by treatment

```
nrow(calcium_tidy)
calcium_tidy |> count(treatment)
```

Step 6: Convert year to an ordered factor

```
calcium_tidy <- calcium_tidy |> mutate( year = factor( year, levels = c("Initial", "Year1",
"Year2", "Year3"), ordered = TRUE ) )
```

Optional: inspect final tidy data

```
head(calcium_tidy) glimpse(calcium_tidy)
```

Step 7: Create a plot with individual trajectories and group mean lines

```
ggplot(calcium_tidy, aes(x = year, y = calcium, group = subject_id)) + geom_line(alpha
= 0.4) + stat_summary( aes(group = 1), fun = mean, geom = "line", linewidth = 1.2 ) +
stat_summary( aes(group = 1), fun = mean, geom = "point", size = 2 ) + facet_wrap(~
treatment) + labs( title = "Ulnar Calcium Levels Over Time by Treatment Group", x = "Year",
y = "Ulnar Calcium Level" ) + theme_minimal()
```

Step 8: Clean theme already added with theme_minimal()

Prompt 2 (Generic Prompt)

Create a data visualization for these data. Calcium.csv

Response: I created a visualization showing the average value of each column in your dataset. Each point represents the mean (average) of one column (y0, y1, y2, y3, etc.). You can quickly compare which variables tend to have higher or lower values overall. For example, some columns cluster around ~70–73, while others drop closer to ~65. If you want something more insightful, I can also create boxplots, time series plots, correlation heatmaps, or side-by-side comparisons.

Prompt 3 (Version Information)

What is your version information?

Response: You're chatting with ChatGPT based on the GPT-5.3 architecture. Model: GPT-5.3. Knowledge cutoff: August 2025. Current date (system): March 31, 2026. # Code Appendix

```
library(dplyr)
library(tidyr)
library(knitr)
library(ggplot2)

airport_data <- tibble(
  airport = c("ATL", "ATL", "ATL", "ATL", "ATL", "ATL",
             "DFW", "DFW", "DFW", "DFW", "DFW", "DFW",
             "DEN", "DEN", "DEN", "DEN", "DEN", "DEN",
             "DXB", "DXB", "DXB", "DXB", "DXB", "DXB",
             "LHR", "LHR", "LHR", "LHR", "LHR", "LHR",
             "PKX", "PKX", "PKX", "PKX", "PKX", "PKX"),
  year = c(2020, 2021, 2022, 2023, 2024, 2025,
           2020, 2021, 2022, 2023, 2024, 2025,
           2020, 2021, 2022, 2023, 2024, 2025,
           2020, 2021, 2022, 2023, 2024, 2025,
           2020, 2021, 2022, 2023, 2024, 2025,
```

```

        2020,2021,2022,2023,2024,2025),
passengers = c(
  42.9,75.7,93.7,104.7,108.1,110.0,
  39.4,62.5,73.4,81.8,87.8,89.7,
  33.7,58.8,69.3,77.8,82.4,84.1,
  25.9,29.1,66.1,86.9,92.3,95.2,
  22.1,28.2,58.2,79.2,83.9,84.0,
  16.0,25.4,13.3,39.4,49.2,53.6
)
)
airport_table <- airport_data |>
  pivot_wider(names_from = year, values_from = passengers)

kable(
  airport_table,
  col.names = c("Airport", "2020", "2021", "2022", "2023", "2024", "2025"),
  caption = "Total Passengers (in millions) for Six Busiest Airports (2020-2025)"
)
airport_data |>
  ggplot(aes(x = year, y = passengers, color = airport)) +
  geom_line(linewidth = 1) +
  geom_point() +
  labs(
    title = "Passenger Trends for Six Busiest Airports",
    x = "Year",
    y = "Passengers (millions)",
    color = "Airport"
  ) +
  theme_minimal()
library(ggplot2)
library(dplyr)

# Monte Carlo simulation function
monte_carlo_sim <- function(n, x_min, x_max, y_min, y_max) {
  x <- runif(n, min = x_min, max = x_max)
  y <- runif(n, min = y_min, max = y_max)
  sim_data <- data.frame(x = x, y = y)
  return(sim_data)
}

# Set the bounds of the rectangular window
x_min <- 0

```

```

x_max <- 4
y_min <- 0
y_max <- 0.8
window_area <- (x_max - x_min) * (y_max - y_min)

# Sample sizes to compare
n_values <- c(100, 500, 1000, 5000)

# Create simulation data for each sample size
all_sim_data <- lapply(n_values, function(n) {
  sim_data <- monte_carlo_sim(
    n = n,
    x_min = x_min,
    x_max = x_max,
    y_min = y_min,
    y_max = y_max
  )

  sim_data |>
    mutate(
      flag = ifelse(
        y > dweibull(x, shape = 1.5, scale = 1),
        "above",
        "on/below"
      ),
      n = paste("n =", n)
    )
}) |>
  bind_rows()

# Create the small multiple plot
ggplot(all_sim_data, aes(x = x, y = y, color = flag)) +
  geom_point(alpha = 0.5, size = 0.8) +
  stat_function(
    fun = dweibull,
    args = list(shape = 1.5, scale = 1),
    xlim = c(0, 4)
  ) +
  facet_wrap(~ n) +
  labs(
    title = "Monte Carlo Integration for Different Sample Sizes",
    x = "x",

```

```

    y = "y",
    color = "Point Type"
  ) +
  theme_minimal()
# Goal:
# Tidy the calcium.csv dataset, wrangle it into a form suitable for visualization,
# and create a plot comparing ulnar calcium levels over time between the
# null and diet/exercise treatment groups.

# Needs:
# calcium.csv, readr, dplyr, tidyr, ggplot2

library(readr)
library(dplyr)
library(tidyr)
library(ggplot2)

# Step 1: Read in the data and inspect it
calcium_raw <- read_csv("calcium (1).csv")

head(calcium_raw)
glimpse(calcium_raw)

# Step 2: Separate the two treatment groups and rename columns
null_group <- calcium_raw |>
  select(1:4) |>
  rename(
    Initial = 1,
    Year1   = 2,
    Year2   = 3,
    Year3   = 4
  )

diet_exercise_group <- calcium_raw |>
  select(5:8) |>
  rename(
    Initial = 1,
    Year1    = 2,
    Year2    = 3,
    Year3    = 4
  )

```

```

# Step 3: Add subject_id and treatment column to each
null_group <- null_group |>
  mutate(
    subject_id = 1:n(),
    treatment = "Null"
  )

diet_exercise_group <- diet_exercise_group |>
  mutate(
    subject_id = 1:n(),
    treatment = "Diet/Exercise"
  )

# Step 4: Pivot each group to long format
null_long <- null_group |>
  pivot_longer(
    cols = Initial:Year3,
    names_to = "year",
    values_to = "calcium"
  )

diet_exercise_long <- diet_exercise_group |>
  pivot_longer(
    cols = Initial:Year3,
    names_to = "year",
    values_to = "calcium"
  )

# Check that rows quadrupled before removing missing values
nrow(null_group)
nrow(null_long)

nrow(diet_exercise_group)
nrow(diet_exercise_long)

# Remove missing values from the null group
# (there is one extra blank row in the null treatment columns)
null_long <- null_long |>
  drop_na(calcium)

# Step 5: Bind the two groups together
calcium_tidy <- bind_rows(null_long, diet_exercise_long)

```



```

# Check total number of rows and counts by treatment
nrow(calcium_tidy)

calcium_tidy |>
  count(treatment)

# Step 6: Convert year to an ordered factor
calcium_tidy <- calcium_tidy |>
  mutate(
    year = factor(
      year,
      levels = c("Initial", "Year1", "Year2", "Year3"),
      ordered = TRUE
    )
  )

# Optional: inspect final tidy data
head(calcium_tidy)
glimpse(calcium_tidy)

# Step 7: Create a plot with individual trajectories and group mean lines
ggplot(calcium_tidy, aes(x = year, y = calcium, group = subject_id)) +
  geom_line(alpha = 0.4) +
  stat_summary(
    aes(group = 1),
    fun = mean,
    geom = "line",
    linewidth = 1.2
  ) +
  stat_summary(
    aes(group = 1),
    fun = mean,
    geom = "point",
    size = 2
  ) +
  facet_wrap(~ treatment) +
  labs(
    title = "Ulnar Calcium Levels Over Time by Treatment Group",
    x = "Year",
    y = "Ulnar Calcium Level"
  ) +
  theme_minimal()

```

```
# Step 8: Clean theme already added with theme_minimal()
```